

GraphOne / FrontierAtlas

AI Data Intelligence Pipeline

Architecture and production-readiness assessment

CURRENT IMPLEMENTATION • LOCAL, DETERMINISTIC, CREDENTIAL-FREE TEST PATH

This document describes the repository as implemented at the time of generation. It separates current local capabilities from external configuration and future distributed scale extensions.

Implemented local pipeline flow

Queue wraps per-record execution



Freshness is applied by source adapters; queueing surrounds the extraction/resolution/persistence worker path.

1. System Overview

The project is a layered Python pipeline for collecting public structured information, normalizing it into canonical records, validating data quality, resolving known entities deterministically, persisting results locally, and exporting reproducible snapshots. The local execution path is asynchronous and bounded; failures are isolated per source/job so one bad record does not terminate an entire batch.

Supported entities

- Startups and Products from deterministic RSS/Atom-style adapters.
- Research Papers from the Semantic Scholar adapter, with optional GitHub repository metadata enrichment.
- Jobs and News from structured public feeds with explicit freshness states.
- Canonical Pydantic records for Startup, Product, Research Paper, Job, and News types.

Implementation boundary

The current implementation uses SQLite, an in-process asyncio queue, injectable Google Sheets clients, and environment-driven provider/Sheets configuration. Real external provider and Google operations are separated from the normal offline test suite.

2. Data Flow

Source acquisition

Async crawler adapters use the shared AsyncHTTPCrawler. A fetch-many operation creates one aiohttp ClientSession for the batch, applies a semaphore, and closes the session deterministically. Startup, Product, Job, and News adapters parse structured feed items; ResearchPaperAdapter uses a structured API response.

Retry and freshness

HTTP retry decisions cover 429 and common 5xx responses, configured timeout/connection failures, and client errors. Delays use exponential backoff with optional jitter; Retry-After is honored for rate limits. Date parsing normalizes ISO, RFC, common textual, relative, and HTML metadata dates. Freshness uses an explicit reference time and a 24-hour boundary: fresh, stale, unknown, and future remain distinct.

Worker execution order

PipelineOrchestrator converts input records into typed PipelineJob instances. AsyncJobQueueExecutor provides bounded workers and retry dispatch. Each successful worker invokes LLM extraction and chunking, Pydantic validation, Startup/Product entity resolution, mapping-log persistence, and canonical SQLite upsert. The Google Sheets exporter runs afterward from SQLite, not directly from crawler output.

3. LLM Architecture

- BaseLLMProvider / LLMProvider defines a generic generate(prompt) contract.
- The configured order is Gemini → Groq → Cerebras; DeepSeek is not part of the implementation.
- Fallback advances only for explicitly classified transient/provider-availability failures: rate limits, timeouts, connection failures, and temporary server errors.
- Authentication/configuration, malformed responses, schema problems, and 413 payload failures are non-retryable at the provider fallback layer.
- Character-based chunking with configurable overlap protects providers from oversized input; chunks are processed with bounded parallelism and merged deterministically.

- The extractor requires JSON-shaped structured output, validates every record with Pydantic, preserves the original source URL, and does not invent missing values.

4. Data Quality

- Source URLs are carried into canonical SourceInfo and used as the normalized SQLite idempotency key.
- Original Startup entityName and Product startupName remain intact while typed entityResolution metadata records original, normalized, canonical, match type, reason, and source URL.
- Entity resolution uses only exact canonical and exact alias matching from deterministic seed data; unknown names remain unresolved.
- Freshness and publication reasons are retained where source models expose them; missing dates are not fabricated.
- Pydantic models reject malformed or incomplete canonical records before persistence.
- Research Paper GitHub enrichment is optional: records without github_url remain valid and github_stars stays absent.

5. Reliability

- AsyncJobQueueExecutor uses a bounded asyncio.Queue and fixed worker count controlled by QUEUE_CONCURRENCY.
- Retryable jobs are routed through a separate retry dispatcher, avoiding workers blocking on retry insertion into a full queue.
- QUEUE_MAX_RETRIES limits attempts; permanent failures are recorded without stopping unrelated jobs.
- Queue metrics include queued, started, succeeded, failed, retried, skipped, cancelled, and duration.
- Producer, retry dispatcher, and workers are cancelled and awaited during cancellation; normal completion drains outstanding jobs before worker shutdown.
- Crawler-level semaphore/backoff and LLM chunk-level limits provide additional backpressure.

6. Storage

SQLite is the current durable local backend. The repository creates the schema explicitly and stores canonical records as JSON plus queryable provenance/freshness columns. `canonical_records` is unique on normalized source URL, so repeated ingestion updates rather than duplicates rows. Bulk writes use a transaction and rollback on failure.

SQLite structure	Role
<code>canonical_records</code>	Canonical Startup, Product, Research Paper, Job, and News payloads plus entity type, source URL, source name, collected/published dates, freshness status, and reason.
<code>entity_mapping_logs</code>	Deduplicated original/normalized/canonical names, match type, reason, and source URL for deterministic entity resolution auditing.

7. Google Sheets

`GoogleSheetsExporter` reads only from `SQLiteRecordRepository` and writes deterministic header-plus-row snapshots through the injectable `GoogleSheetsClient` interface. The official `GoogleSheetsAPIClient` is an adapter around Google's Sheets API and service-account authentication, while fake clients keep unit tests offline.

- Required tabs: Startups; Products; Research Papers; Jobs; News; Entity Mapping Log.
- Rows are sorted deterministically and source URLs, freshness fields, and resolution metadata are included where applicable.
- Each export clears/replaces the tab snapshot, preventing duplicate rows on repeated exports and leaving empty datasets as header-only tabs.
- `GOOGLE_SHEETS_SPREADSHEET_ID` and `GOOGLE_SHEETS_CREDENTIALS` are external configuration only. Credentials, tokens, private keys, and JSON contents must never be committed.
- No live Google export is claimed in this document; the normal test suite uses fake clients and the live test is separately marked/skipped when configuration is absent.

8. Scale to 500k+ Records

Current implementation

The current system is a local bounded architecture: one process, in-memory async queue, fixed workers, bounded retries, SQLite persistence, and deterministic snapshot export. It is suitable for reproducible development, assessment, and small-to-medium local batches; it is not a distributed 500k-record deployment.

Future production extensions

The `PipelineJob/JobResult` boundary can later be transported through Redis, Kafka, Celery, or an equivalent queue without changing crawler, extraction, resolution, or persistence contracts. Horizontal workers could partition work by source, entity type, or stable job key. A persistent production database would replace SQLite behind the repository interface. Batching, rate-limit-aware scheduling, caching, normalized-URL deduplication, monitoring/metrics, and retry/dead-letter handling would be added around the same provenance and idempotency rules.

9. Security / Responsible Crawling

- Adapters target public structured feeds/APIs and do not bypass CAPTCHAs or anti-bot controls.
- Crawler concurrency and retry behavior are bounded and should respect source restrictions and rate limits.
- Provider, GitHub, and Google credentials are read through environment/configuration; no credential values are embedded in source or documentation.

- The repository's normal tests do not make external network calls and do not require credentials.

10. Testing

The offline test strategy uses fake HTTP responses, fake LLM providers, fake Google Sheets clients, temporary SQLite databases, and deterministic reference times. Coverage includes crawler retries, freshness boundaries, chunking, provider fallback, entity resolution, queue cancellation/retry behavior, persistence, export idempotency, and end-to-end mixed-success processing.

Latest verified result: 207 passed, 2 deselected. The deselected tests are live external-integration tests. Compilation with `python -m compileall -q src` tests passed. No live provider or Google Sheets operation is claimed as part of this verification.

11. Implementation Status

Implemented

Async public-source crawling; retries/backoff/jitter/Retry-After; freshness/date normalization; Startup/Product/Research Paper/Job/News models and adapters; Gemini → Groq → Cerebras extraction fallback; chunking and Pydantic validation; deterministic entity resolution; optional GitHub enrichment; SQLite persistence and mapping logs; bounded batch/job queue; injectable and official Google Sheets export adapters; end-to-end offline validation.

External configuration required

Live LLM calls require the corresponding provider environment variables. Live Google export requires `GOOGLE_SHEETS_SPREADSHEET_ID`, `GOOGLE_SHEETS_CREDENTIALS`, Google Sheets API enablement, and spreadsheet sharing with the service account. These values are intentionally absent from the repository.

Future scaling extensions

Distributed queue transport, horizontal workers, persistent production database, partitioning/sharding, centralized rate limiting, caching, monitoring, and durable retry/dead-letter workflows.

Prepared from the current repository implementation. This document is architectural documentation, not a claim of live external-service execution.